CS F342 COMPUTER ARCHITECTURE

ASSIGNMENT 1

Implement a 6-stage pipelined processor in Verilog. This processor supports load (lw), store (sw), jump (j), subtract (sub), and NAND immediate (nandi) instructions only. The processor should implement forwarding to resolve data hazards. The processor has Reset, CLK as inputs and no outputs. The processor has instruction fetch unit, decode, reg read (with 32 32-bit registers), execution, memory and writeback units. The processor also contains five pipelined registers IF/ID, ID/RR, RR/EX, EX/MEM and MEM/WB. When reset is activated the PC, IF/ID, ID/RR, RR/EX, EX/MEM and MEM/WB registers are initialized to 0, the instruction memory and register file get loaded by predefined values.

When the instruction unit starts fetching the first instruction the pipelined registers contain unknown values. When the second instruction is being fetched in the IF unit, the IF/ID register will hold the instruction code for the first instruction. When the third instruction is being fetched by the IF unit, the IF/ID register contains the instruction code of the second instruction, the ID/RR register contains information related to the first instruction and so on. (Assume a 32-bit PC. Also Assume Address and Data size as 32-bit).

The instruction and its 32-bit instruction format are shown below:

lw destinationReg, offset [sourceReg] (Sign extends data specified in instruction field (15:0) to 32-bits, add it with register specified by register number in rs field and store the data corresponding to the memory location defined by the calculated address in the rt register. Opcode for lw is 100011).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

sw sourceReg, offset [destinationReg] (Signextends data specified in instruction field (15:0) to 32-bits, add it with register specified by register number in rs field. Opcode for sw is 101011).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

sub destinationReg, sourceReg 1, sourceReg 2 (Perform sub operation on the registers specified by registers specified by register numbers in rs and rt fields and save the result in the register specified by register specified by register number in rd field. Opcode for sub is 000000, shamt is 00000 and function is 100010).

op	rs	rt	rd	shamt	funct
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	5-bits (15-11)	5-bits (10-6)	6-bits (5-0)

nandi destinationReg, sourceReg, immediate (Sign extends data specified in instruction field (15:0) to 32-bits, NAND it with register specified by register number in rs field. And store the result in rt. Opcode for nandi is 001110).

op	rs	rt	offset
6 bits (31-26)	5-bits (25-21)	5-bits (20-16)	16-bits (15-0)

j target (Jumps to an address generated by appending 00 to the right and 4 higher order bits of the program counter to the left, of the 26 bit address field in the instruction. Opcode for j is 000010).

op	address
6 bits (31-26)	26-bits (25-0)

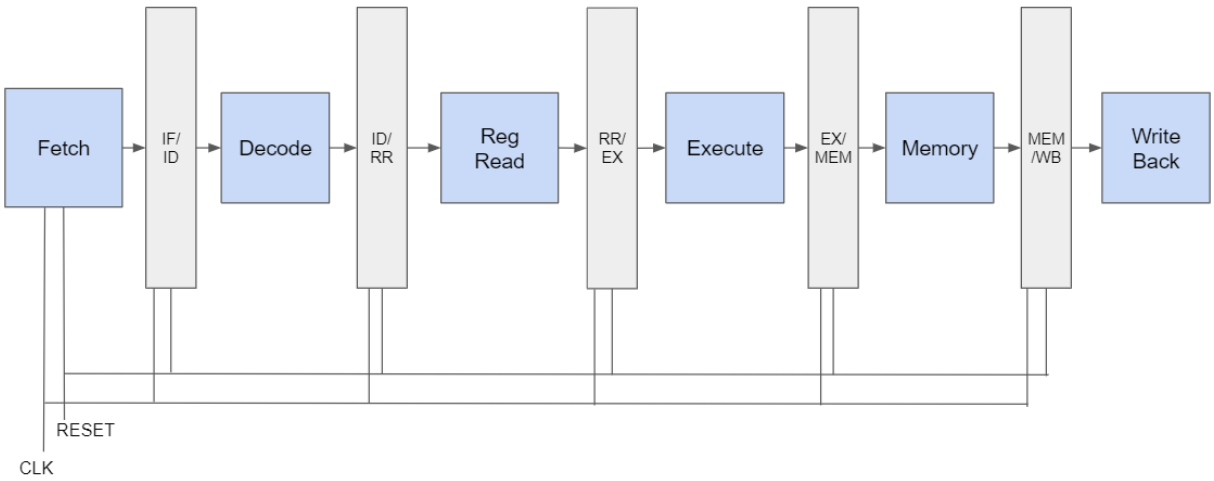
Assume the register file contains 32 registers (R0-R31) each register can hold 32-bit data. On reset, PC and all register file registers should get initialized to 0. Ensure r0 is always zero. Each location in DMEM has 8-bit data. So, to store a 32-bit value, you need 4 locations in the DMEM, stored in big-endian format. Also ensure that on reset, the instruction memory gets initialized with the following instructions, starting at address 0:

```
nandi R3, R8, #8
sub R6, R6, R6
lw R2, R1, #12
sub R5, R3, R7
```

```
        add R4, R2, R5
        j     L1
        sub R3, R4, R5
L1: sub R1, R2, R11
```

The above code should run correctly on the processor implementation. Ensure that you handle the data hazards present, if any.

A partial block level representation of the 6-stage pipelined processor is shown below. Please note that for registerfile implementation, write should be on the positive edge and read should be on the negative edge of the clock. Write operation depends on the control signal.



As part of the assignment three files should be submitted in a zipped folder.

1. PDF version of this Document with all the Questions below answered with file name as IDNO_NAME.pdf.
2. Design Verilog Files for all the Sub-modules (instruction fetch, Register file, forwarding unit).
3. Design Verilog file for the main processor.

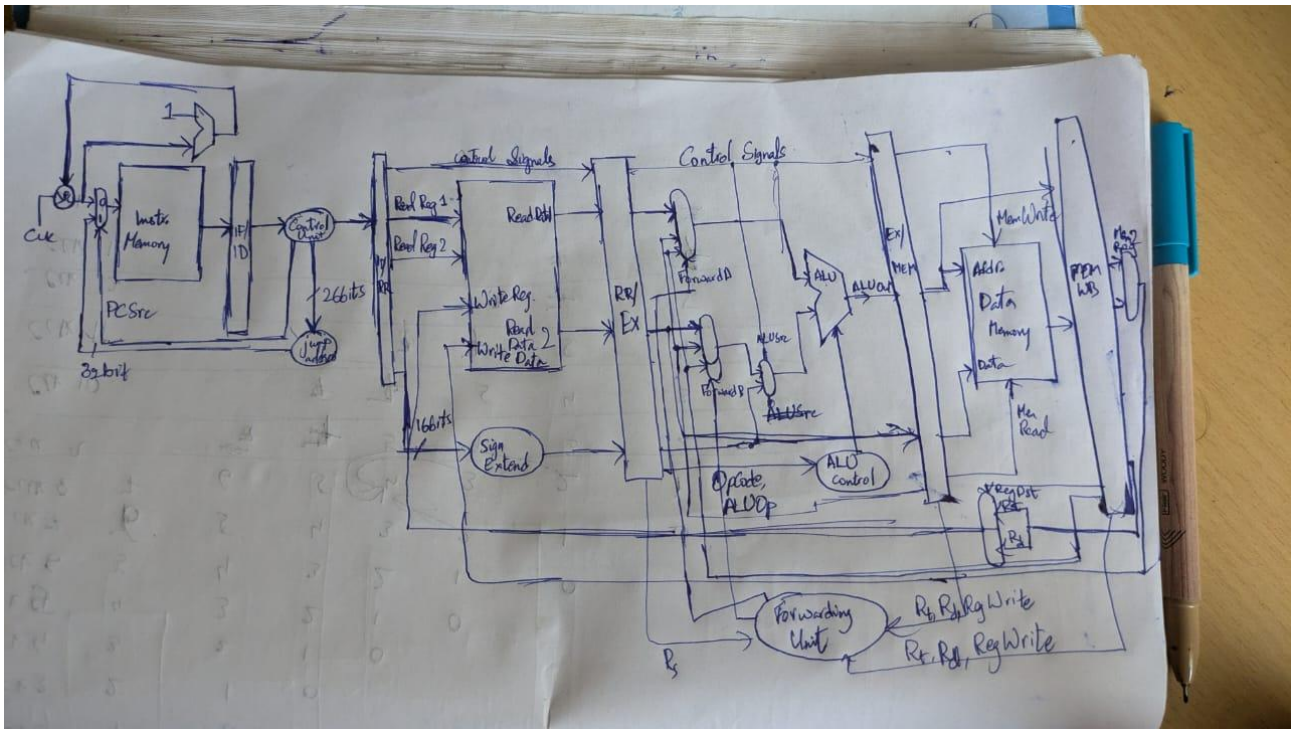
The name of the zipped folder should be in the format IDNO_NAME.zip

NAME: Harshith Kambampati ID No: 2022A3PS0252G

Questions Related to Assignment

1. Draw the complete Datapath and show control signals of the 6-stage pipelined processor. The reference slide uploaded can be used to edit according to your specifications.

Answer:



2. List the control signals used and also the values of control signals for different instructions in a tabular format as follows:

Answer:

	sw	lw	sub	nandi	jump.
PCSrc	0	0	0	0	1
RegWrite	1	0	1	1	~~x~~ (0)
ALUSrc	1	1	0	1	~~x~~ (0)
ALUOp	Add 00	Add 00	Subtraction 01	Nandi 11	x (00)
MemRead	0	0	0	0	x (0)
MemWrite	0	1	0	0	x (0)
Mem2Reg	0	x	1	1	x (0)
RegDst	1	0	0	~~x~~	x (0)

3. In a program, 30% of the instructions are load instructions, and 1/x of them incur a stall due to being immediately followed by an instruction that uses their result. Additionally, 10% of the instructions are store instructions, while 45% are R-type instructions. There are also 10% branch instructions, with 1/y of them being taken. Finally, 5% of the instructions are jump instructions. Given a total of 10^9 instructions and a clock cycle time of 100 picoseconds, what is the average CPI of this program? Furthermore, assuming a MIPS single-cycle pipelined processor always predicts branches as not taken, how much time does it take to execute all instructions?

Where x, y, z are related to last 3 digits of your ID No.

If ID number: 20XXXXXXABCG, then x = (A % 8) + 1, y = ((B + 2) % 8) + 1, and z= ((C + 3) % 8) + 1.

Answer: Average CPI:1.15, Total Time taken: 0.115sec

4. Implement the Instruction Fetch block. Copy the image of Verilog code of the Instruction fetch block here

Answer:

```
module Instruction_memory(
input clk,
input rst,
input [31:0] PC,
output [31:0] instruction
);

reg [31:0]imem[31:0];
always @(*)
begin
if(rst)
begin
imem[0] = 32'h39030008;
imem[1] = 32'h88C63000;
imem[2] = 32'h8C22000C;
imem[3] = 32'h88672800;
imem[4] = 32'h88452001;
imem[5] = 32'h08000006;
imem[6] = 32'h88832806;
imem[7] = 32'h08000000;
imem[24] = 32'h884A0800;

end

end
assign instruction = imem[PC];
endmodule
```

5. Implement the Instruction Decode block. Copy the image of Verilog code of the Instruction decode block here

Answer:

```
module Instruction_Decode(
    input [31:0] instruction_code,
    output [5:0] opcode,
    output [4:0] R_s,
    output [4:0] R_t,
    output [4:0] R_d,
    output [15:0] offset,
    output [4:0] shamt,
    output [5:0] function_code,
    output [25:0] address
    );


    assign opcode = instruction_code[31:26];
    assign R_s = instruction_code[25:21];
    assign R_t = instruction_code[20:16];
    assign R_d = instruction_code[15:11];
    assign offset = instruction_code[15:0];
    assign shamt = instruction_code[10:6];
    assign function_code  = instruction_code[5:0];
    assign address = instruction_code[25:0];

endmodule
```

6. Determine the condition that can be used to detect data hazard?

Answer:

if (EX_MEM_Rd == RR_EX_Rs && EX_MEM_regWrite !=0) forward = 01

else if (MEM_WB_Rd = RR_EX_Rs && EX_MEM_regWrite !=0) forward = 10

else forward = 00

Similarly, for Rt in place of Rs register

7. Implement the Register File and copy the image of Verilog code of Register file unit here.

Answer:

```
module Register_File(input rst,input clk,input [4:0] Read_Reg_1,
    input [4:0] Read_Reg_2,input [4:0] Write_Reg,
    input [31:0] Write_Data,input Reg_Write,output [31:0] Read_Data_1,
    output [31:0] Read_Data_2
    );
    reg [31:0]registers
    always @(*)
    begin
        if(rst)
            begin
            registers[0] =32'h0;registers[1] =32'h4;registers[2] =32'h2;registers[3] =32'h3;
            registers[4] =32'h41;registers[5] =32'h5;registers[6] =32'h6;registers[7] =32'h7;
            registers[8] =32'h8;registers[9] =32'h9;registers[10] =32'ha;registers[11] =32'hb;
            registers[12] =32'hc;registers[13] =32'hd;registers[14] =32'he;registers[15] =32'hf;
            registers[16] =32'h10;registers[17] =32'h11;registers[18] =32'h12;registers[19] =32'h13;
            registers[20] =32'h14;registers[21] =32'h15;registers[22] =32'h16;registers[23] =32'h17;
            registers[24] =32'h18;registers[25] =32'h19;registers[26] =32'h1a;registers[27] =32'h1b;
            registers[28] =32'h1c;registers[29] =32'h1d;registers[30] =32'h1e;registers[31] =32'h1f;
            end
    end
    always @(posedge clk)begin
    if(Reg_Write)begin
    registers[Write_Reg] = Write_Data;
    end
    end
    assign Read_Data_1 = registers[Read_Reg_1];
    assign Read_Data_2 = registers[Read_Reg_2];
endmodule
```

output [31:0]

8. Implement the forwarding unit and copy the image of Verilog code of forwarding unit here.

Answer:

```
module forward_unit(
    input [4:0] R_t_exmem,input [4:0] R_d_exmem,input rd_exmem,
    input regwrite_exmem,input [4:0] R_t_memwb,
    input [4:0] R_d_memwb,input regwrite_memwb,input rd_memwb,
    input [4:0] R_s_ex,input [4:0] R_t_ex,output reg[1:0] forward_A,
    output reg[1:0] forward_B
    );
    wire [5:0]EX_MEM_Rd;wire [5:0]MEM_WB_Rd;
    assign EX_MEM_Rd = rd_exmem? R_t_exmem : R_d_exmem;
    assign MEM_WB_Rd = rd_memwb? R_t_memwb : R_d_memwb;
    always @(*) begin
    if(EX_MEM_Rd == R_s_ex && regwrite_exmem !=0 && EX_MEM_Rd != 5'b00000)
    begin
    forward_A <= 2'b01;
    end
    else if(MEM_WB_Rd == R_s_ex && regwrite_memwb !=0 && MEM_WB_Rd != 5'b00000)
    begin
    forward_A <= 2'b10;
    end
    else begin forward_A = 2'b00; end
    if(EX_MEM_Rd == R_t_ex && regwrite_exmem !=0 && EX_MEM_Rd != 5'b00000)
    begin
    forward_B <= 2'b01;
    end
    else if(MEM_WB_Rd == R_t_ex && regwrite_memwb !=0 && MEM_WB_Rd != 5'b00000)
    begin
    forward_B <= 2'b10;
    end
    else begin forward_B = 2'b00; end
    end
endmodule
```

9. Implement a complete processor in Verilog (using all the Datapath blocks). Copy the image of Verilog code of the processor here. (Use comments to describe your Verilog implementation)

Answer:

```
module Processor(input clk,input rst);
    reg [31:0]IF_ID;reg [49:0]ID_RR;reg [129:0]RR_EX;reg [78:0]EX_MEM;reg [76:0]MEM_WB; // Stage Registers
    reg [31:0]PC =32'h0;wire rst_pin;wire [31:0] instruction;wire [5:0]opcode;wire [4:0]R_s;wire [4:0]R_t;wire [4:0]R_d;
    wire [15:0]offset;wire [31:0]immediate_data;wire RegWrite;wire ALU_Src; wire[1:0]ALUOp;
    wire memread;wire memwrite;wire memtoreg;wire PCSrc;wire regDst; wire [31:0]instr_out;wire [4:0]shamt;wire[25:0]address;wire [5:0]func_code;
    // Instruction Fetch Stage
    wire [31:0]j_address;
    jump_address ja(IF_ID,address,j_address);Instruction_memory im(clk,rst,PC,instruction);
    // Instruction Decode Stage
    assign instr_out = IF_ID;Instruction_Decode id(instr_out,opcode,R_s,R_t,R_d,offset,shamt,func_code,address);
    Control_Unit c_u(opcode,RegWrite,ALU_Src,ALUOp,memread,memwrite,memtoreg,PCSrc,regDst);
    // Register Read Stage
    wire [31:0]Reg_Data_1;wire [31:0]Reg_Data_2; wire [4:0]R_Dest;wire [31:0]result; wire [15:0]off;wire [5:0]rs;wire [5:0]rt;wire rw;wire rd;
    assign rs = ID_RR[49:45];assign rt = ID_RR[44:40];
    assign off = ID_RR[34:19];assign rw = MEM_WB[11];
    assign rd = MEM_WB[10]; //regdst signal to select between rt and rd at write back stage
    wire rt1; assign rt1 = MEM_WB[9:5];assign R_Dest = rd ? MEM_WB[4:0] :MEM_WB[9:5];
    Register_File r_f(rst,clk,rs,rt,R_Dest,result,rw,Reg_Data_1,Reg_Data_2);Sign_extend se(ID_RR[34:19],immediate_data);
    // Execute stage
    wire [3:0]ALU_Control; wire [1:0]aluop;wire [5:0]fc;assign aluop = RR_EX[17:16];assign fc = RR_EX[28:23];wire [1:0]forward_A;wire [1:0]forward_B;
    forward_unit fu(EX_MEM[9:5],EX_MEM[4:0],EX_MEM[10],EX_MEM[11],MEM_WB[9:5],MEM_WB[4:0],MEM_WB[10],MEM_WB[11],RR_EX[129:125],RR_EX[9:5],forward_A,forward_B);
    ALU_control alu_c(aluop,fc,ALU_Control);reg [31:0]ALU_Input_1;wire [31:0]ALU_Input_2;wire [31:0]ALU_out;wire ALU_zero;wire [31:0]Data_out;reg [31:0] B_rf;
    assign ALU_Input_2 = RR_EX[15] ? RR_EX[60:29] : B_rf;ALU alu(ALU_Input_1,ALU_Input_2,ALU_Control,ALU_out,ALU_zero);
    //Memory Stage
    wire [31:0]addr ; wire memw =EX_MEM[14];wire memr = EX_MEM[13];wire [31:0]writedata;
    assign writedata = EX_MEM[78:47];assign addr = EX_MEM[46:15];Data_Memory data_mem(rst,memr,memw,addr,writedata,Data_out);
    // Write Back Stage
    assign result = MEM_WB[12] ?MEM_WB[76:45] : MEM_WB[44:13];
    always @(posedge clk)
    begin if(rst) begin PC = 32'b0;IF_ID=32'b0;ID_RR=50'b0;RR_EX =130'b0;EX_MEM=79'b0;MEM_WB=77'b0; end
    else begin if(PCSrc)begin PC <= j_address;end else begin PC <=PC+1;end
    IF_ID <= instruction;ID_RR <= {R_s,R_t,R_d,offset,func_code,shamt,ALUOp,ALU_Src,memread,memwrite,memtoreg,RegWrite,regDst};
    RR_EX <= {ID_RR[49:45],Reg_Data_1,Reg_Data_2,immediate_data ,ID_RR[18:0],ID_RR[44:35]};EX_MEM <= {RR_EX[92:61],ALU_out,RR_EX[14:0]};
    MEM_WB <= {EX_MEM[46:15],Data_out,EX_MEM[12:0]};
    end   end
    always @(*)begin
    case(forward_A) 2'b00:ALU_Input_1 <= RR_EX[124:93];2'b01:ALU_Input_1 <= EX_MEM[46:15];
    2'b10:case(MEM_WB[12])
        1'b0: ALU_Input_1 <= MEM_WB[44:13];1'b1:ALU_Input_1 <= MEM_WB[76:45];
    endcase endcase
    case(forward_B) 2'b00:B_rf <= RR_EX[92:61]; 2'b01:B_rf <= EX_MEM[46:15];
    2'b10:case(MEM_WB[12])
        1'b0: B_rf <= MEM_WB[44:13];1'b1: B_rf <= MEM_WB[76:45];
        endcase  endcase end endmodule
```

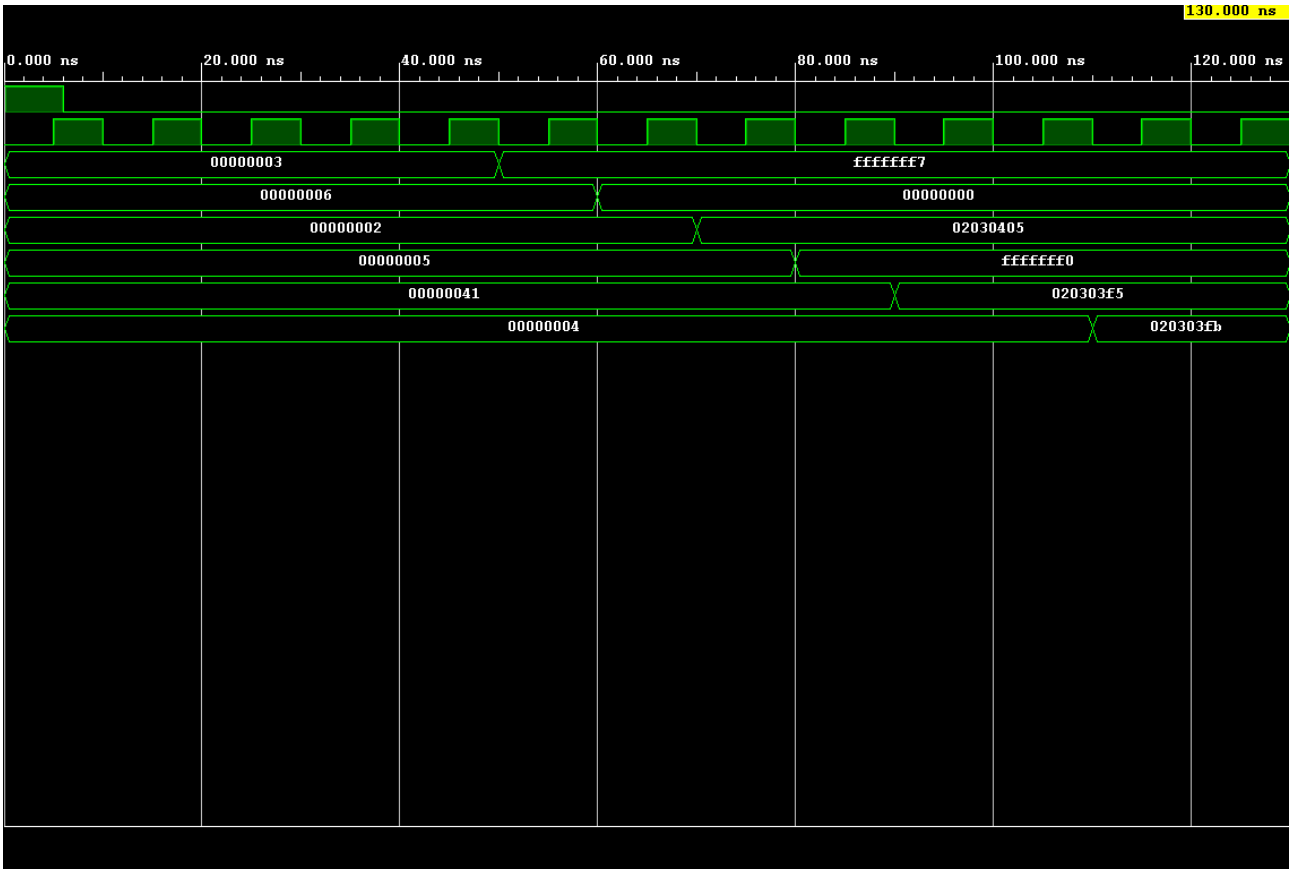
10. Test the processor design by generating the appropriate clock and reset. Copy the image of your testbench code here.

Answer:

```
module tb(

    );
reg rst;
reg clk;
Processor p(clk,rst);

initial begin #0 clk =0;#0 rst =1; #6 rst =0;end
initial begin
repeat (26) #5 clk =~clk; $finish; end



endmodule
```

11. Verify if the register file is getting updated according to the set of instructions (mentioned earlier).

Copy verified Register file waveform here (show only the Registers that get updated, CLK, and RESET):

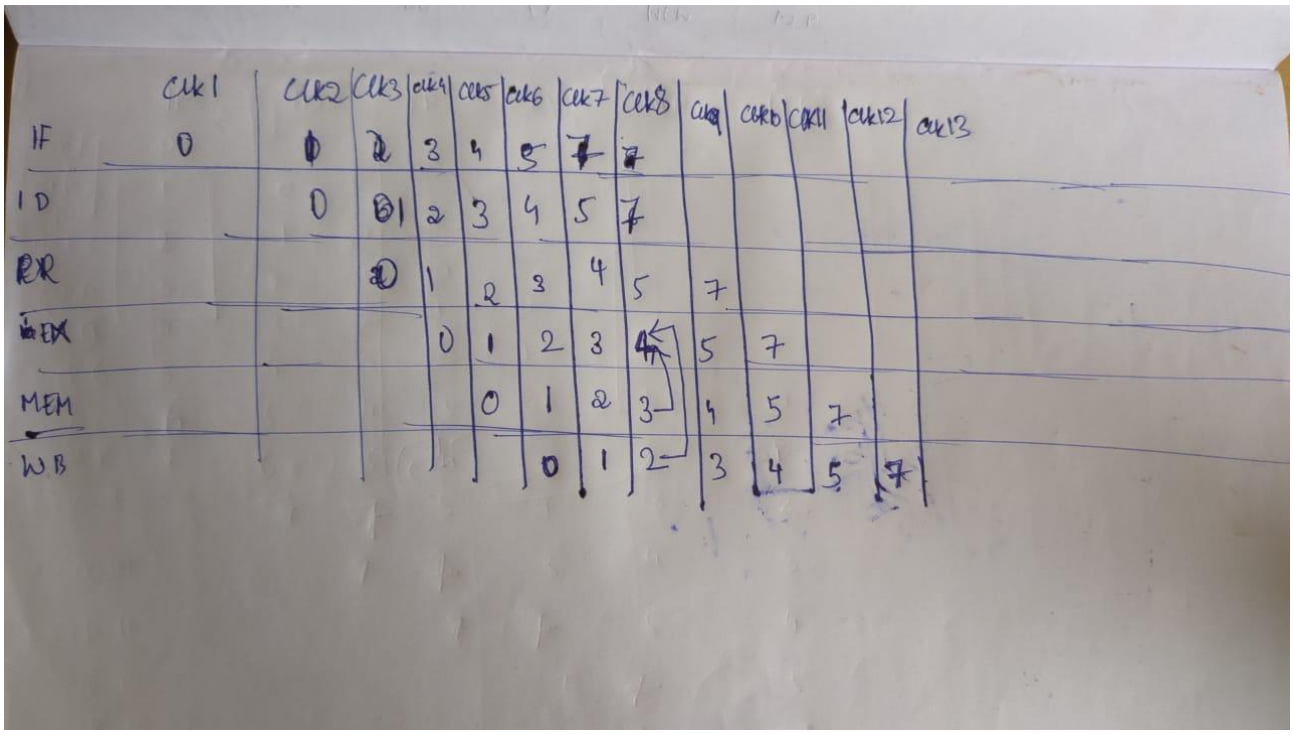


12. What is the total number of cycles needed to issue the program given above on the pipelined MIPS Processor? What is the CPI of the program?

Answer: 13

13. Make a diagram showing the clock by clock execution of each instruction, indicating stalling, forwarding etc wherever necessary.

Answer:



	clk1	clk2	clk3	clk4	clk5	clk6	clk7	clk8	clk9	clk10	clk11	clk12	clk13
IF	0	1	2	3	4	5	7	7					
ID		0	1	2	3	4	5	7					
RR			0	1	2	3	4	5	7				
EX				0	1	2	3	4	5	7			
MEM					0	1	2	3	4	5	7		
WB						0	1	2	3	4	5	7	

14. Is your design synthesizable? Which target FPGA was used for synthesis?

Answer: Yes. It was synthesizable for xc7z020clg484-1

15. Provide the synthesis report in tabular form (resources consumed)?

Answer:

```
Start RTL Component Statistics
---------------------------------------------------------------
Detailed RTL Component Info :
+---Adders :
       3 Input   32 Bit       Adders := 1
       2 Input   32 Bit       Adders := 1
+---Registers :
                  130 Bit    Registers := 1
                   79 Bit    Registers := 1
                   77 Bit    Registers := 1
                   50 Bit    Registers := 1
                   32 Bit    Registers := 1
+---Muxes :
       2 Input   32 Bit        Muxes := 8
       4 Input   32 Bit        Muxes := 4
       4 Input    8 Bit        Muxes := 566
       2 Input    8 Bit        Muxes := 1062
       5 Input    8 Bit        Muxes := 6
       2 Input    5 Bit        Muxes := 3
       6 Input    4 Bit        Muxes := 1
       5 Input    3 Bit        Muxes := 2
       4 Input    2 Bit        Muxes := 60
       5 Input    2 Bit        Muxes := 1
       2 Input    1 Bit        Muxes := 1087
       5 Input    1 Bit        Muxes := 761
       4 Input    1 Bit        Muxes := 259
       3 Input    1 Bit        Muxes := 1
       6 Input    1 Bit        Muxes := 1
---------------------------------------------------------------
Finished RTL Component Statistics
---------------------------------------------------------------
```

Unrelated Questions

What were the problems you faced during the implementation of the processor?

Answer: The implementation of forwarding unit, designing the pipelined registers and control unit.

Did you implement the processor on your own? If you took help from someone whose help did you take? Which part of the design did you take help for?

Answer: Yes. I took help from Chandrahas.

Honor Code Declaration by student:

- My answers to the above questions are my own work.
- I have not shared the codes/answers written by me with any other students. (I might have helped clear doubts of other students).
- I have not copied other's code/answers to improve my results. (I might have got some doubts cleared from other students).

Name: Harshith Kambampati Date:

ID No.: 2022A3PS0252G 14-04-2025